Modulo 4 · Lezione 8: MCP, collegare il coding agent e l'assistente alle app

MCP (Model Context Protocol) è il modo standard con cui un'applicazione AI usa i tool di un sistema che non è suo. Oggi lo vedete con le app che usa l'ufficio del personale di Valtesa: Trello, Notion, Airtable. Il fornitore dell'app scrive e tiene acceso un server MCP; voi scrivete il suo indirizzo, e il login lo fa il client. Lo collegate a due client: il coding agent, con il connettore di claude.ai, e l'assistente, con l'Agents SDK. Per il codice vostro, invece, basta quello che conoscete già: un function tool.

Preparare la cartella

All'inizio della lezione, o al più tardi prima della parte sull'assistente, dalla radice della vostra copia della repo del corso (la cartella che contiene `modulo4-agenti/`):

```
git switch main
git pull
git switch -c l08-lavoro
```

Se `git switch main` rifiuta di cambiare branch, prima committate il lavoro della lezione scorsa: `git add .` e `git commit -m "lavoro della lezione scorsa"`.

Preparare l'ambiente

Sempre dalla radice della repo:

```
cd modulo4-agenti/l08_mcp/scripts/working
source ../../../.venv/bin/activate
cp ../../../l07_skill/scripts/working/.env .env
pytest
```

Su PowerShell il venv si attiva con `..\..\..\.venv\Scripts\Activate.ps1` e la chiave si copia con `copy ..\..\..\l07_skill\scripts\working\.env .env`. Se la cartella della lezione scorsa non ha il `.env`: copiate `.env.example` in `.env` e incollate la chiave del corso dopo `OPENAI_API_KEY=`.

`pytest` deve dire `91 passed` (su Windows senza i permessi per i link simbolici: `90 passed, 1 skipped`). I test non chiamano le API e non vanno in rete. Per l'esercizio non c'è niente da installare: la libreria `mcp` e l'Agents SDK sono già nel venv.

I file della lezione

File	A cosa serve
`working/`	il progetto Valtesa: la soluzione del docente di Modulo 4 · Lezione 7, con la skill `report-mensile`

File	A cosa serve
`working/app_agente.py`	l'assistente con il function tool `note_fuori_massimale` e il server MCP di un'app (Trello, Notion o Airtable)
`working/login_mcp.py`	il login OAuth all'app, come lo fa Claude Code: salva il token in `.mcp-token/`
`working/esercizio.md`	dove scrivete l'app scelta e le due righe dell'esercizio
`working/tests/test_app_agente.py`	i test del function tool e del file del token
`working/.mcp-token/`	i token dei vostri login: non c'è finché non fate il login, ed è nel `.gitignore`

Le parole del login

- **OAuth**: il modo standard per dare a un programma l'accesso al vostro account di un'app senza dargli la password.
- **Token**, due significati. Nel login è la chiave che il client riceve alla fine e manda a ogni richiesta: chi ce l'ha agisce come voi. Nel resto del corso sono i pezzi di testo che il modello legge e che si pagano. Oggi servono tutti e due: si capisce dal contesto.
- **Codice di autorizzazione**: quello che il fornitore rimanda al client dopo che avete acconsentito nel browser. Vale pochi minuti e una volta sola: il client lo scambia con il token.
- **localhost e porta**: `localhost` è il vostro computer; `8765` è la porta dove `login_mcp.py` ascolta il ritorno del browser. Da fuori non si raggiunge.
- **401 e 400**: risposte HTTP. 401 vuol dire «non autorizzato»: manca il token, o non vale più. 400 vuol dire «richiesta sbagliata».
- **Server MCP e server del login**: il server MCP è quello del fornitore, in cloud, con i tool. Il piccolo server web su `localhost:8765` è di `login_mcp.py`, e serve solo ad aspettare il codice: si spegne a login finito.

Il codice vostro e le app degli altri

Nella lezione scorsa lo stesso conto, il massimale in vigore in un giorno, stava in due posti: in Python in `rimborsi.py`, per il coding agent, e scritto in italiano nella skill dell'assistente. Il conto è codice vostro, e il codice vostro sapete già come darlo ai due client:

- all'**assistente** un function tool, come in Modulo 3 · Lezione 10:

```
@function_tool
def note_fuori_massimale(mese: str) -> str:
    """Le note spese fuori massimale di un mese AAAA-MM, calcolate dal programma dei
rimborsi: …"""
    return testo_fuori_massimale(mese)
```

`testo_fuori_massimale` legge `dati/note_spese.csv` e usa `controlla` e `limite_nota` di `rimborsi.py`: il conto lo fa il programma, nel processo dell'assistente;

- al **coding agent** lo script: `python rimborsi.py`, lanciato con `Bash`, come fa la skill `report-mensile`.

Per il codice vostro non serve un protocollo nuovo. Serve quando il codice è di un altro: le app che l'ufficio usa ogni giorno, una board Trello per le note da verificare, un database Notion, una base Airtable, la posta. Lì il codice e l'account sono del fornitore. Potreste scrivere un function tool che chiama le API di Trello, ma dovreste studiarle, gestire il login e tenerlo aggiornato. Il fornitore invece pubblica un **server MCP**, a un indirizzo, e lo tiene acceso: voi scrivete l'indirizzo.

App	Indirizzo del server
Trello	`https://mcp.trello.com/v1`
Notion	`https://mcp.notion.com/mcp`
Airtable	`https://mcp.airtable.com/mcp`
Gmail, Google Calendar, Google Drive	dai connettori di claude.ai

Si può anche scrivere un server MCP, con la stessa libreria `mcp` che avete nel venv: oggi non lo facciamo. Conviene quando lo stesso codice lo usano tanti client, o client che non controllate voi (l'agente di un'altra squadra). Per `rimborsi.py` i client sono due, e sono vostri: Claude Code lo lancia con `Bash`, l'assistente lo chiama con un function tool di poche righe.

Cos'è MCP

La definizione ufficiale, dalla pagina «What is the Model Context Protocol» di https://modelcontextprotocol.io:

> MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.
>
> Think of MCP like a USB-C port for AI applications.

MCP è uno standard open source per collegare le applicazioni AI a sistemi esterni; funziona come una porta USB-C, un attacco solo per tanti dispositivi. Le applicazioni AI si chiamano **client** (Claude Code, Codex, Cursor, il vostro assistente); i sistemi esterni hanno ognuno il suo **server**. Chi fa un'app scrive il server una volta, e lo usano tutti i client.

Un server MCP può offrire tre cose: **tool**, funzioni che l'applicazione chiama; **risorse**, dati da leggere; **prompt**, modelli di richiesta da riempire. Oggi usiamo solo i tool.

Tool del client e tool MCP

Fin qui i tool stavano dentro il client che li usa. Claude Code ha i suoi (`Read`, `Bash`, `Edit`, …), nel suo processo; l'assistente ha i suoi function tool (`leggi_file`, `cerca`, `elenca_file`, e oggi `note_fuori_massimale`), nel processo dell'assistente. Un tool così lo usa solo il client che lo contiene.

Un **tool MCP** sta in un altro processo, il server. `trelloWriteCard`, che crea una card, sta nel server di Trello, in cloud. Ogni client che si collega al server lo aggiunge ai suoi tool. Per il modello sono uguali: in tutti i casi vede nome, descrizione e parametri, e chiede il tool.

E perché non uno script, o un function tool che chiama le API? Lo script funziona per il coding agent, che ha `Bash`, ma il modello deve sapere quale script lanciare e come leggerne l'output; l'assistente `Bash` non ce l'ha. Un function tool che chiama le API di Trello lo scrivete e lo mantenete voi, login compreso, e lo usa solo l'assistente. Il server MCP lo scrive e lo aggiorna il fornitore, e lo usano tutti i client.

Il modello chiede, il client inoltra, il server esegue

Con MCP gli attori sono tre:

1. il **client** (Claude Code o l'assistente) manda al modello la domanda e l'elenco dei tool, compresi quelli del server;
2. il **modello**, in rete, sceglie un tool e scrive gli argomenti: «chiama `trelloWriteCard`, nella lista Da verificare, con la nota 2377»;
3. il client vede che è un tool del server e gli **inoltra** la chiamata (`tools/call`);
4. il **server**, in cloud dal fornitore, **esegue**: crea la card e rimanda il risultato;
5. il client passa il risultato al modello, che scrive la risposta.

Server locali e in cloud

Un server MCP può girare sul vostro computer (**locale**: il client lo avvia e gli parla su stdin e stdout) o dal fornitore (**in cloud**: il client si collega a un indirizzo, in HTTPS). Oggi sono tutti in cloud.

	Locale	In cloud
Chi lo avvia	il client, quando parte	il fornitore: è sempre acceso
Dove stanno i dati	sul vostro computer	dal fornitore; al server arrivano gli argomenti del modello
Come si entra	nessun login	con il vostro account: il login lo fa il client

Il protocollo è lo stesso: le stesse richieste, `tools/list` e `tools/call`.

Il login lo fa il client

Un server in cloud vuole sapere chi siete. Il login lo fa il client, con **OAuth**; il modello non vede mai password né token.

1. Il client chiama il server senza credenziali; il server risponde **401** e dice dove leggere come si fa il login. La risposta vera di Trello:

```
POST https://mcp.trello.com/v1  →  401
www-authenticate: Bearer resource_metadata="https://mcp.trello.com/.well-known/oauth-protected-resource/v1"
```

2. Il client **si registra** presso il fornitore, da solo (registrazione dinamica del client): non dovete creare un'app sul sito di Trello.

3. Si apre il **browser**: entrate con il vostro account e acconsentite. Il browser torna dal client con un **codice di autorizzazione**: per Claude Code e per l'assistente su un indirizzo `localhost`, dove il client aspetta; per il connettore su claude.ai.

4. Il client scambia il codice con un **token**, lo salva e lo manda a ogni richiesta; quando scade, lo rinnova.

Con il connettore di claude.ai questi passi li fa claude.ai, una volta. Con `claude mcp add` li fa Claude Code, con `/mcp`. Nell'assistente li fa `login_mcp.py`.

Il token è una **chiave del vostro account**: chi ce l'ha agisce come voi. Si tiene fuori da git (`.mcp-token/` è nel `.gitignore`), non si incolla in una chat, e si revoca dalle impostazioni dell'app, tra le applicazioni collegate.

La registrazione dinamica non la accettano tutti. Il 21/10/2026 l'abbiamo provata su una quarantina di servizi: la accettano quasi tutti; Google (Gmail, Calendar, Drive), Slack, HubSpot, Box e Smartsheet chiedono un'app registrata a mano sul sito del fornitore, e dal coding agent si usano con i connettori di claude.ai.

I messaggi tra client e server

I messaggi sono JSON-RPC: un metodo, dei parametri, una risposta. Quelli veri tra l'assistente e Trello, registrati dal lato del client, senza le intestazioni con il token (`→` parte dall'assistente, `←` risponde il server):

```
→ server/discover {}
← 400 «Request must be an initialize request if no session ID is provided.»
→ initialize {"protocolVersion": "2025-11-25", …}
← 200 {"serverInfo": {"name": "atlassian-mcp-server", …}}
→ tools/list {}
← 200 15 tool: trelloReadCard, trelloReadBoard, … trelloWriteCard, …
→ tools/call {"name": "trelloReadBoard", "arguments": {"action": "list", "limit": 100}}
← 200 {"content": [{"type": "text", "text": "{ \"boards\": [ … ] }"}], "isError": false}
```

- **La versione.** Il client prova la versione più nuova del protocollo, del luglio 2026: comincia con `server/discover`, che chiede al server chi è e cosa sa fare. Il server di Trello parla quella di prima e risponde 400, e il client ripiega su `initialize`, con la versione 2025-11-25, l'ultima che comincia così.
- **`tools/list`**: l'elenco dei tool, con nome, descrizione e parametri, una volta.
- **`tools/call`**: una chiamata per ogni tool che il modello chiede.

Cosa riceve il modello

La prima richiesta che l'assistente manda all'API di OpenAI, accorciata: i tool di Trello stanno accanto ai nostri, nella stessa forma dei function tool.

```
"tools": [ … elenca_file, cerca, leggi_file, note_fuori_massimale,
  {"name": "trelloReadBoard",
   "parameters": {"type": "object", "properties": {
      "action": {"type": "string", "enum": ["list", "list_by_workspace", "get", …],
                 "description": "Action to perform. \"list\" — paginated boards the
authenticated user is a member of …"},
      …}},
   "strict": false, "type": "function", "description": "Read Trello boards …"}, …]
```

Le descrizioni le scrive il fornitore: lunghe e in inglese, e si pagano in token a ogni chiamata. In Claude Code lo stesso tool si chiama `mcp__claude_ai_Trello__trelloReadBoard` (mcp, il connettore, il tool) e arriva **differito**: all'inizio solo il nome, e la definizione la carica `ToolSearch` quando serve, come i tool differiti di Modulo 4 · Lezione 1.

Il modello non parla con il server: sceglie il tool e scrive gli argomenti; i messaggi MCP li scambiano client e server.

Collegare un'app al coding agent

Il modo più semplice è il **connettore di claude.ai** (come si collega: passo 1 dell'esercizio). Nell'app di Claude, scheda Code, il connettore si aggiunge alla sessione dal pulsante **+** accanto al campo di testo, alla voce dei connettori. Da terminale, se siete entrati con l'abbonamento, Claude Code i connettori di claude.ai li vede da solo: `/mcp` li elenca, con davanti «claude.ai», e `claude mcp list` dà:

```
claude.ai Trello: https://mcp.trello.com/v1 - ✔ Connected
```

Per controllare, nell'app e da terminale, chiedete nella sessione: «quali connettori hai in questa sessione?». Da terminale Claude Code vede i connettori solo con il login dell'abbonamento, non con una chiave API: `/status` dice quale login è attivo. Con l'app su WSL i connettori possono mancare: lì usate Claude Code da terminale, cioè `claude` lanciato in `working/`; la prima volta `/login`, con l'account dell'abbonamento.

Se usate Claude Code da terminale e preferite non usare il connettore:

```
claude mcp add --transport http trello https://mcp.trello.com/v1
```

e poi, nella sessione, `/mcp` per il login nel browser.

Nella nostra prova (Claude Code 2.1.289, Sonnet 5.5, 21/10/2026), alla richiesta di mettere in Trello le note fuori massimale di luglio, Claude Code ha letto `rimborsi.py` e l'ha lanciato con `Bash` (il conto), ha caricato i tool di Trello con `ToolSearch`, ha cercato le board per non toccare quelle che c'erano, e ha creato board, lista e 20 card: 52 secondi, 29 turni. Era in modalità Auto, dove i permessi li dà un classificatore, senza chiedere. In Manual Claude Code chiede il permesso per ogni tool dell'app, anche per quelli che leggono.

Collegare un'app all'assistente

Nell'assistente il collegamento e il login sono codice, già scritto in `app_agente.py` e `login_mcp.py`.

```
APP = {"trello": {"url": "https://mcp.trello.com/v1",
                  "tool": ["trelloReadWorkspace", "trelloReadBoard", "trelloWriteBoard",
                           "trelloReadList", "trelloWriteList", "trelloWriteCard"],
                  "strutturato": False}, …}

server = MCPServerStreamableHttp(
    name=nome,
    params={"url": app["url"], "auth": autenticazione(nome, app["url"]), "timeout": 60},
    tool_filter=create_static_tool_filter(allowed_tool_names=app["tool"]),
    cache_tools_list=True,
    use_structured_content=app["strutturato"],
)
agente = assistente_base.clone(instructions=assistente_base.instructions + "\n" +
ISTRUZIONI_APP…,
                               tools=assistente_base.tools + [note_fuori_massimale],
mcp_servers=[server])
```

- `MCPServerStreamableHttp` è il client MCP dell'SDK per i server in cloud (per quelli locali c'è `MCPServerStdio`).
- `auth` è l'oggetto che fa il login e aggiunge il token a ogni richiesta, da `login_mcp.py`.
- `tool_filter` tiene solo i tool che servono: il server di Trello ne ha 15, Airtable 47, Notion 65, e l'SDK manda le definizioni di tutti a ogni chiamata. Il filtro è per tool, non per azione: Trello non ha tool che cancellano, ma `trelloWriteCard` e `trelloWriteList` sanno anche modificare e archiviare. Per questo `ISTRUZIONI_APP` dice di non toccare quello che c'è.
- `use_structured_content` serve ad Airtable, che risponde solo con dati strutturati: senza, l'SDK passa al modello risposte vuote, e il modello dice che non avete workspace.
- `ISTRUZIONI_APP` è una riga in più nelle istruzioni (sotto il perché).

`login_mcp.py`. Il client OAuth è nella libreria `mcp` (`OAuthClientProvider`): `login_mcp.py` gli dà il file del token (`.mcp-token/<app>.json`), il modo di aprire il browser e un piccolo server web su `localhost:8765` che aspetta il ritorno del browser. `fai_login` fa solo il login, **prima** della sessione MCP, con `httpx2` (il client HTTP della libreria `mcp`, già nel venv): dentro la sessione la prima richiesta ha un timeout di circa un minuto, e scadrebbe mentre siete nel browser (il browser tornerebbe su `localhost` e troverebbe «connection refused»). Il login si fa una volta per app: se il token non c'è, il primo `python app_agente.py <app> "…"` apre il browser prima di collegarsi; le volte

dopo usa il token salvato. `python app_agente.py <app> --login` fa solo il login, e se il token c'era lo butta e lo rifà: serve anche quando il token non vale più. Il login del connettore non vale qui: quel token lo tiene claude.ai, e lo usa solo Claude Code. Per ogni app i login sono due, uno per client. Il token resta nella cartella dove avete fatto il login e non si copia altrove: in un'altra cartella si rifà il login, con `--login`.

Le istruzioni. Le istruzioni dell'assistente dicono che lavora sulla cartella dell'ufficio, che ha tre tool (`elenca_file`, `cerca`, `leggi_file`) e che rifiuta le domande che non riguardano Valtesa: dell'app non sanno niente. `ISTRUZIONI_APP` è una riga in più: l'ufficio usa anche quell'app, si legge e si crea senza toccare quello che esiste, i conti li fa `note_fuori_massimale`. La stessa domanda, «Nell'app ci sono già spazi dell'ufficio del personale di Valtesa? Dimmi i nomi, senza modificare niente.», una prova per app (21/10/2026):

App	Le istruzioni di sempre	Con `ISTRUZIONI_APP`
Trello	trova le board di Valtesa	trova le board di Valtesa
Notion	cerca nella cartella dei file, non in Notion	trova la pagina
Airtable	guarda solo i workspace e risponde «non c'è»	trova la base

Con una domanda generica («come si chiamano le mie board?») e le istruzioni di sempre ha risposto «Non posso rispondere a questa richiesta», su tutte e tre le app. Quando date a un agente un tool nuovo, rileggete le istruzioni: dove cercare, cosa si può fare.

Esercizio: le note da verificare nella vostra app (45')

File: `working/esercizio.md`, da scrivere; `working/app_agente.py` si lancia, non si modifica. Claude Code e un terminale con il venv attivo, tutti e due aperti in `working/`.

Le note spese fuori massimale di luglio 2026, una voce per nota, nell'app che scegliete: prima dal coding agent, poi dall'assistente.

Variante	Dove finiscono le note	Comando per l'assistente	Da sapere
Trello	una board, una lista «Da verificare», una card per nota	`python app_agente.py trello`	15 tool; una card per volta; provata con tutti e due i client
Notion	una pagina con un database «Da verificare», una riga per nota	`python app_agente.py notion`	65 tool; nel consenso scegliete la pagina di prova; a volte crea un database vuoto in più, che si cancella a mano; con Claude Code non l'abbiamo provata
Airtable	una base con una tabella «Da verificare», un record per nota	`python app_agente.py airtable`	47 tool; dati strutturati, già gestito in `app_agente.py`; con Claude Code non l'abbiamo provata

Valori di controllo: a luglio 2026 le note fuori massimale sono 20, su 164; tra queste la 2377 di Luca Ferri, cena a Roma il 28 luglio, 46,42 € su 45 €.

Mettete Claude Code in **Manual**: nell'app dal selettore accanto al tasto di invio; da terminale è la modalità di partenza, e Shift+Tab la cambia. In Manual chiede il permesso per ogni tool dell'app, anche per quelli che leggono; in Auto decide da solo. A quelli che leggono o creano rispondete sì; a uno che cancella o modifica quello che c'era, no. Nella richiesta di permesso ci sono anche gli argomenti: con Trello `"action": "create"` crea, `"update"` e `"archive"` modificano. Per il tool che crea le voci (`trelloWriteCard`, `notion-create-pages`, `create_records_for_table`) scegliete di non farvi chiedere più, o sono 20 domande: da lì non chiede più neanche per le altre azioni dello stesso tool (con `trelloWriteCard` anche modificare e archiviare), ed è per questo che si lavora in uno spazio nuovo. Sì anche a lanciare `rimborsi.py` o comandi `python` in `working/`.

1. **L'app.** Un account gratuito su Trello, Notion o Airtable; se ne usate già una, va bene quella. Se non sapete quale: Trello, l'unica che abbiamo provato con tutti e due i client. Con Notion create una pagina di prova vuota, «Valtesa · prova»: al login sceglierete quella.

   Poi il connettore, su claude.ai, con l'account di Claude dell'abbonamento che usate per Claude Code: **Impostazioni → Connettori → Sfoglia**, cercate l'app, **Connetti**, e fate il login all'app; se vi chiede a cosa dare accesso, scegliete un workspace o la pagina di prova. Se il connettore di Airtable non c'è, scegliete Trello o Notion.

   Nella sessione di Claude Code chiedete «quali connettori hai in questa sessione?»: ci deve essere l'app. Nell'app di Claude, se manca, aggiungetela dal **+** accanto al campo di testo; da terminale `/mcp` la mostra con davanti «claude.ai». Scrivete in `esercizio.md` l'app scelta.

2. **Claude Code.** Una sessione in `working/`:

   > Metti in Trello le note spese fuori massimale di luglio 2026: crea una board nuova «Valtesa · note da verificare», con una lista «Da verificare» e una card per ogni nota, con id, dipendente, voce, città, giorno, importo e massimale. Il conto fallo con il programma dei rimborsi di questa cartella. Non modificare niente di quello che c'è già.

   Con Notion:

   > Metti in Notion le note spese fuori massimale di luglio 2026: dentro la pagina «Valtesa · prova» crea una pagina nuova «Valtesa · note da verificare», con un database «Da verificare» e una riga per ogni nota, con id, dipendente, voce, città, giorno, importo e massimale. Il conto fallo con il programma dei rimborsi di questa cartella. Non modificare niente di quello che c'è già.

   Con Airtable:

   > Metti in Airtable le note spese fuori massimale di luglio 2026: crea una base nuova «Valtesa · note da verificare», con una tabella «Da verificare» e un record per ogni nota, con id, dipendente, voce, città, giorno, importo e massimale. Il conto fallo con il programma dei rimborsi di questa cartella. Non modificare niente di quello che c'è già.

   Nella sessione guardate quali tool chiama, e in che ordine. Se parte la skill `report-mensile` e si mette a scrivere il report del mese, fermatelo (Esc) e chiedete solo il conto con `rimborsi.py`.

3. **Il login dell'assistente.** Nel terminale, in `working/` e con il venv attivo (se l'avete chiuso: `source ../../../.venv/bin/activate`):

   ```
   python app_agente.py trello --login
   ```

   (o `notion`, `airtable`). Si apre il browser: entrate e acconsentite. Il login è fatto quando il terminale scrive «Login a trello fatto»; se scrive «Login non riuscito», leggete il motivo e rilanciate. Se il browser non si apre, copiate nel browser l'indirizzo stampato nel terminale.

4. **L'assistente.** La stessa richiesta, con un nome nuovo per lo spazio. Il programma dei rimborsi e «non modificare niente» non servono: li dicono già le istruzioni dell'assistente (`ISTRUZIONI_APP`).

   ```
   python app_agente.py trello "Metti in Trello le note spese fuori massimale di luglio 2026: crea una board nuova «Valtesa · note dall'assistente», con una lista «Da verificare» e una card per ogni nota, con id, dipendente, voce, città, giorno, importo e massimale."
   ```

   Con Notion:

   ```
   python app_agente.py notion "Metti in Notion le note spese fuori massimale di luglio 2026: dentro la pagina «Valtesa · prova» crea una pagina nuova «Valtesa · note dall'assistente», con un database «Da verificare» e una riga per ogni nota, con id, dipendente, voce, città, giorno, importo e massimale."
   ```

   Con Airtable:

   ```
   python app_agente.py airtable "Metti in Airtable le note spese fuori massimale di luglio 2026: crea una base nuova «Valtesa · note dall'assistente», con una tabella «Da verificare» e un record per ogni nota, con id, dipendente, voce, città, giorno, importo e massimale."
   ```

   Per circa un minuto non stampa niente: lavora. Alla fine stampa la traccia: cercate `note_fuori_massimale` e poi i tool dell'app.

5. **Le due righe.** In `esercizio.md`: chi ha fatto il conto e chi ha scritto nell'app, per il coding agent e per l'assistente, come lo leggete nella sessione e nella traccia.

Fatto quando nell'app ci sono le 20 note di luglio due volte, una dal coding agent e una dall'assistente, in due spazi; nella traccia dell'assistente c'è `note_fuori_massimale` prima del tool dell'app che scrive; le due righe sono in `esercizio.md`. Poi un commit, da `working/`:

```
git add .
git commit -m "le note di luglio nell'app"
```

`.mcp-token/` resta fuori dal commit: git la ignora. Controllate con `git status` che non ci sia.

Se qualcosa non va.

Cosa vedete	Cosa fare
Il browser dice che `localhost` rifiuta la connessione	il programma non era più in ascolto: rilanciate `python app_agente.py <app> --login` e rifate il login
«La porta 8765 è occupata»	c'è un altro login aperto in un altro terminale: chiudetelo (Ctrl+C) e rilanciate
«Login non riuscito: …»	nel browser non avete acconsentito, o il fornitore ha dato un errore: rilanciate con `--login`
Il terminale resta fermo dopo il login nel browser	Ctrl+C e rilanciate con `--login`
Mentre l'assistente lavora si apre il browser del login, o compaiono errori 401	il token non vale più (revocato, o scaduto da tanto): Ctrl+C, poi `--login`, che lo butta e rifà il login
«Il giro si è fermato dopo 15 chiamate al modello»	guardate nell'app cosa ha già creato, e chiedete di finire in quello spazio, senza crearne un altro
In WSL il browser non si apre	copiate l'indirizzo stampato nel browser di Windows: il ritorno su `localhost:8765` arriva a WSL (provato su WSL2)
PowerShell rifiuta di eseguire `Activate.ps1`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, poi di nuovo l'attivazione

Estensioni, per chi finisce prima:

- **Una seconda app**, tra le tre.
- **Un'app non provata.** Dovrebbero funzionare con lo stesso codice. Qui `app_agente.py` si modifica: si aggiunge l'app in `APP`, con l'indirizzo, `"tool": []` e `"strutturato": False`, poi `python app_agente.py <app> --login` e `python app_agente.py <app> --tool`, che stampa i tool del server; i nomi che servono vanno in `"tool"`. Todoist (`https://ai.todoist.net/mcp`), ClickUp (`https://mcp.clickup.com/mcp`), Monday (`https://mcp.monday.com/mcp`), Jira e Confluence (`https://mcp.atlassian.com/v1/mcp`), Miro (`https://mcp.miro.com/`), Linear (`https://mcp.linear.app/mcp`), Asana (`https://mcp.asana.com/sse`, che usa il trasporto vecchio: nell'SDK `MCPServerSse` al posto di `MCPServerStreamableHttp`). Non le abbiamo provate: scrivete cosa succede.
- **Una bozza di mail.** Con il connettore Gmail di claude.ai, dal coding agent: «prepara una bozza di mail all'ufficio con il riepilogo delle note fuori massimale di luglio». Solo una bozza: all'invio si risponde no.
- **Un server di documentazione.** Context7 dà al coding agent la documentazione aggiornata delle librerie, senza account. Da terminale: `claude mcp add --transport http context7 https://mcp.context7.com/mcp`. Collegato da solo, nelle nostre prove Claude Code non lo usava; con questa riga in `AGENTS.md` sì: «Prima di scrivere codice che usa una libreria, cerca la documentazione della versione installata con il server MCP context7.»
- **Il costo.** La domanda «Nell'app ci sono già spazi dell'ufficio del personale di Valtesa? Dimmi i nomi, senza modificare niente.» con e senza `tool_filter` (in `server_app` togliete la riga `tool_filter=…`): la traccia stampa i token in ingresso di tutto il giro.

Rientro: cosa è successo (da leggere dopo l'esercizio)

Chi ha calcolato, chi ha scritto. Con Trello:

Client	Chi ha fatto il conto	Chi ha scritto nell'app
Claude Code	`rimborsi.py`, lanciato con `Bash`	`trelloWriteCard`, eseguito dal server di Trello
L'assistente	`note_fuori_massimale`: `rimborsi.py`, nel processo dell'assistente	`trelloWriteCard`, eseguito dal server di Trello

Il conto è codice vostro, e sta dove sta il codice vostro; le card sono dell'app, e le scrive il server del fornitore. Il modello sceglie i tool e scrive gli argomenti.

Quanto costa un'app nel contesto dell'assistente (`gpt-6-luna`, 21/10/2026, con `tool_filter`):

App	Tool del server	Tenuti	Chiamate al modello	Token in ingresso
Trello, 20 card scritte	15	6	6	circa 63.000
Airtable, 20 record scritti	47	6	5	circa 34.000
Notion, una domanda di sola lettura	65	4	4	circa 35.000

L'SDK manda le definizioni dei tool a ogni chiamata, e quelle dei fornitori sono lunghe: nella prima richiesta a Trello le definizioni dei tool sono 24.262 caratteri su 25.565. Claude Code invece le carica solo quando servono, con `ToolSearch`. Conta quanti tool tenete e quanto sono lunghe le loro definizioni, non quanti ne ha il server: i quattro di Notion sono circa 23.700 caratteri, i sei di Trello circa 21.900. E in ogni chiamata c'è anche la storia, che cresce con i risultati dei tool. Notion è misurata su una domanda di sola lettura: i numeri della sua scrittura non li abbiamo registrati.

Permessi e chiavi. Un server di un'app agisce con il vostro account. In Claude Code il controllo è il permesso, tool per tool, in Manual. Nell'assistente nessuno chiede niente, a meno che lo mettiate voi: `tool_filter` per non dare i tool che non servono, e `require_approval`, un parametro del server MCP nell'SDK: per i tool che indicate il giro si ferma e aspetta un sì prima di eseguirli. In `app_agente.py` c'è solo `tool_filter`. Il token è una chiave: fuori da git, mai in una chat.

Dopo questa lezione

La prossima lezione è Modulo 4 · Lezione 9, un lab: mettete in pratica le skill e MCP, con Claude Code e con l'Agents SDK.

Prima della prossima lezione: un commit del vostro lavoro sul branch `108-lavoro`, poi dalla radice della repo:

```
git switch main
git pull
```